Klausur — Themeneingrenzung

Genau diese Konzepte kommen in der Klausur dran

HTW Berlin – Wirtschaftsingenieurwesen
WiSe 2026/27

Worum es hier geht

Diese Folien grenzen den **Klausurstoff** ein.

- Pro Thema: die Konzepte, die **tatsächlich geprüft** werden – jeweils mit einem **kurzen Beispiel**.
- Jede Folie nennt das **Notebook & den Foliensatz**, in dem ihr das Thema vertiefen könnt.
- Es gibt **zwei Klausurteile**: *Programme* (Python) und *Datenbanken* (SQL).

Wichtig: Konzepte, die hier **nicht** auftauchen, sind **nicht** Teil der Klausur. Lernt gezielt das, was auf diesen Folien steht.

Nicht klausurrelevant – Pandas & Matplotlib

Notebook	Thema	Status
16a / 16b	Pandas: Visualisierungen, DataFrames	optional
17	Pandas: Statistik	optional
18 / 19	Matplotlib: Grundlagen & Vertiefung	optional

- **Optionales Selbststudium** – nicht in der Vorlesung behandelt.
- Freiwilliges Zusatz-Übungsblatt: *Pandas Visualisierungen*.
- Im Berufsalltag sehr nützlich – ansehen lohnt sich, **muss aber niemand**.

Klar gesagt: Pandas und Matplotlib kommen **weder im Multiple-Choice-Teil noch in den Programmieraufgaben** vor.

Teil 1 – Programme (Python)

Geprüft werden Grundlagen der Programmierung bis hin zum strukturierten Programm:

- **Grundlagen:** Konsole, Kommentare, Variablen, Datentypen
- **Datenstrukturen:** Listen, Tupel, Dictionaries
- **Logik & Ablauf:** Funktionen, Operatoren, Verzweigungen, Schleifen
- **Robustheit & Struktur:** Fehlerbehandlung, `main()`-Programmaufbau

Stoff: Notebooks 01–11 & 15 – entsprechende Foliensätze.

Einführung – Grundbegriffe

Konzept	Beispiel / Bedeutung
Programmiersprache	Python: `print("Hallo")`
Python-Interpreter	führt Code Zeile für Zeile aus
Algorithmus	Schritt-für-Schritt-Anleitung zum Ziel
Syntax vs. Semantik	`prnt()` = Syntaxfehler; falsches Ergebnis = Semantik
Fehlerarten	Syntax-, Laufzeit-, logischer Fehler

Stoff: Notebook 01 & Foliensatz 01 – *Einführung in die Programmierung*.

Konsole – Ein- und Ausgabe

Konzept	Beispiel
`print()`	`print("Gesamtwert:", 2228.30)`
`input()`	`name = input("Artikel: ")`
f-Strings	`print(f"{preis:.2f} €")`

Stoff: Notebook 02 & Foliensatz 02 – *Konsole*.

Kommentare

Konzept	Beispiel
Einzeiliger Kommentar `#`	`# berechnet den Gesamtwert`
Mehrzeiliger Kommentar	`"""Beschreibung über mehrere Zeilen"""`

Stoff: Notebook 03 & Foliensatz 03 – *Kommentare*.

Variablen

Konzept	Beispiel
Variable & Zuweisung =	`bestand = 120`
Initialisierung	`summe = 0.0`
Namens-Regeln (snake_case)	`laptop_staender` (kein Start mit Ziffer)

Stoff: Notebook 04 & Foliensatz 04 – *Variablen*.

Einfache Datentypen

Konzept	Beispiel
`str` (Zeichenkette)	`name = "Eco-Sneaker"`
`int` (Ganzzahl)	`anzahl = 35`
`float` (Gleitkomma)	`preis = 49.90`
`bool` (Wahrheitswert)	`versandfrei = True`
`type()`	`type(preis)  # <class 'float'>`
Typumwandlung	`int("50")  # ergibt 50`
`input()` liefert `str`	`menge = int(input("Menge: "))`

Stoff: Notebook 05 & Foliensatz 05 – *Einfache Datentypen*.

Komplexe Datentypen – Listen & Tupel

Konzept	Beispiel
Liste `[]`	`namen = ["USB-Kabel", "Webcam"]`
Index-Zugriff	`namen[0]  # "USB-Kabel"`
`append()`	`namen.append("Funkmaus")`
`remove()`	`namen.remove("Webcam")`
`len()`	`len(namen)  # 2`
Tupel `()`	`groesse = (4.99, 120)` (unveränderlich)

Stoff: Notebook 06 & Foliensatz 06 – *Komplexe Datentypen*.

Komplexe Datentypen – Dictionaries

Konzept	Beispiel
Dictionary `{k: v}`	`preise = {"Webcam": 49.90}`
Zugriff `d["k"]`	`preise["Webcam"]  # 49.90`
Wert anlegen/ändern	`preise["Webcam"] = 45.00`
`keys()` / `values()`	`preise.keys()`

Stoff: Notebook 06 & Foliensatz 06 – *Komplexe Datentypen*.

Funktionen

Konzept	Beispiel
Definition def	`def gesamtwert(bestaende, preise):`
Aufruf	`gesamtwert(bestaende, preise)`
Parameter	`def versand(summe):`
Benannte Argumente	`versand(summe=60)`
`return`	`return summe`
Scope (lokal/global)	Werte über **Parameter**, nicht global

Stoff: Notebook 07 & Foliensatz 07 – *Funktionen*.

Operatoren

Konzept	Beispiel
Arithmetik `+ - * /`	`summe = summe + anzahl * preis`
Vergleich `== != < > <= >=`	`bestand < 10`
`and`	`summe >= 20 and summe < 50`
`or`	`eingabe == "fertig" or fertig`
`in` / `not in`	`name in bestaende`
Priorität & Klammern	`(a + b) * c`

Stoff: Notebook 08 & Foliensatz 08 – *Operatoren*.

Programmverzweigungen

Konzept	Beispiel
`if`	`if summe >= 50: return 0.0`
`if` / `else`	`else: return 4.95`
`elif`	`elif summe >= 20: return 2.95`

Stoff: Notebook 09 & Foliensatz 09 – *Programmverzweigungen*.

Schleifen

Konzept	Beispiel
`while`	`while menge < 1:`
`while True` + `break`	`while True: ...; break`
`for`	`for name in namen:`
`for` über `dict.items()`	`for name, anzahl in bestaende.items():`
`break`	`if eingabe == "fertig": break`
`continue`	`if name not in katalog: continue`

Stoff: Notebook 10 & Foliensatz 10 – *Schleifen*.

Fehlerbehandlung

Konzept	Beispiel
`try` / `except`	`try: int(x) … except ValueError:`
`ValueError`	`int("abc")  # ValueError`
weitere Fehlertypen	`10 / 0  # ZeroDivisionError`
mehrere except-Blöcke	`except ValueError: … except TypeError:`
`finally`	`finally: conn.close()`
robuste Eingabe	`while True:` + `try: return int(input(...))`

Stoff: Notebook 11 & Foliensatz 11 – *Fehlerbehandlung*.

Strukturierte Programme

Konzept	Beispiel
Programmstruktur	Imports, Konstanten, Funktionen, `main()`
`main()`-Funktion	`def main(): ...`
Funktionen orchestrieren	`main()` ruft `gesamtwert()`, `versand()` …
Konstanten (GROSS)	`MINDESTBESTAND = 10`
Docstring	`"""Berechnet den Gesamtwert."""`
Single Responsibility	eine Funktion = eine Aufgabe
`return` statt `print`	`return summe` (Aufrufer entscheidet)

Stoff: Notebook 15 & Foliensatz 15 – *Strukturierte Programme*.

Teil 2 – Datenbanken (SQL)

Geprüft wird der Weg von der Modellierung zur fertigen SQLite-Datenbank:

- **Grundlagen:** Was ist eine Datenbank / ein DBMS?
- **Modellierung:** relationales Modell, Schlüssel, Beziehungen, ER-Diagramm
- **SQLite mit Python:** Verbindung, Tabellen anlegen, Daten einfügen & abfragen
- **Abfragen vertiefen:** Filter, Sortierung, JOINs, Aggregatfunktionen

Stoff: Notebooks 20–23 – entsprechende Foliensätze.

Datenbanksysteme – Grundbegriffe

Konzept	Beispiel / Bedeutung
Datenbank-Begriff	strukturierte Datensammlung in Tabellen
DBMS	Software dahinter, z. B. SQLite, PostgreSQL
Excel vs. Datenbank	DB: Multi-User, Konsistenz, große Mengen
Einsatzbereiche	Online-Shop, ERP, CRM, Banking
ACID (Überblick)	Atomicity, Consistency, Isolation, Durability

Stoff: Notebook 20 & Foliensatz 20 – *Datenbanksysteme*.

Relationale Datenbanken – Modell & Schlüssel

Konzept	Beispiel / Bedeutung
Relationales Modell	Daten in Tabellen plus Beziehungen
Tabelle / Zeile / Spalte	kunde: Zeile = Datensatz, Spalte = Attribut
Primärschlüssel	`kunde_id INTEGER PRIMARY KEY`
PK-Eigenschaften	eindeutig, nicht NULL, unveränderlich
Fremdschlüssel	`FOREIGN KEY (kunde_id) REFERENCES kunde`
Referentielle Integrität	FK verhindert verwaiste Bestellungen
NULL-Semantik	NULL = unbekannt, nicht `0` oder `""`
`IS NULL`	`WHERE email IS NULL`

Stoff: Notebook 21 & Foliensatz 21 – *Relationale Datenbanken*.

Relationale Datenbanken – Beziehungen & Datentypen

Konzept	Beispiel / Bedeutung
1:1-Beziehung	eine Person hat genau einen Pass
1:n-Beziehung	ein Kunde hat viele Bestellungen
n:m-Beziehung	viele Produkte in vielen Bestellungen
Zwischentabelle	löst eine n:m-Beziehung auf
ER-Diagramm	Kästen = Tabellen, Linien = Beziehungen
numerische Typen	`INTEGER`, `DECIMAL`, `FLOAT`
Text-Typen	`CHAR`, `VARCHAR`, `TEXT`
Datums-/Zeit-Typen	`DATE`, `TIME`, `DATETIME`

Stoff: Notebook 21 & Foliensatz 21 – *Relationale Datenbanken / ERD*.

SQLite – Verbindung & Arbeitsablauf

Konzept	Beispiel
SQLite	Datenbank = eine `.db`-Datei
`sqlite3`-Modul	`import sqlite3`
`connect()`	`conn = sqlite3.connect("veggie_soles.db")`
`cursor()`	`cursor = conn.cursor()`
`commit()`	`conn.commit()`
`close()`	`conn.close()`
with-Context Manager	`with sqlite3.connect("x.db") as conn:`

Stoff: Notebook 22 & Foliensatz 22 – *SQLite-Grundlagen*.

SQLite – Tabellen anlegen

Konzept	Beispiel
`CREATE TABLE`	`CREATE TABLE kunde (...)`
`IF (NOT) EXISTS` / `DROP`	`DROP TABLE IF EXISTS kunde`
SQLite-Datentypen	`INTEGER`, `REAL`, `TEXT`, `BLOB`, `NULL`
`PRIMARY KEY`	`kunde_id INTEGER PRIMARY KEY`
`NOT NULL`	`name TEXT NOT NULL`
`UNIQUE`	`mitarbeiter_id INTEGER UNIQUE`
`FOREIGN KEY`	`FOREIGN KEY (kunde_id) REFERENCES kunde`

Stoff: Notebook 22 & Foliensatz 22 – *SQLite-Grundlagen*.

SQLite – Daten einfügen

Konzept	Beispiel
`INSERT`	`INSERT INTO kunde VALUES (?, ?, ?, ?)`
`execute()`	`cursor.execute(sql)`
Parametrisierung `?`	`cursor.execute(sql, (1, "Anna"))`
`executemany()`	`cursor.executemany(sql, datenliste)`

Stoff: Notebook 22 & Foliensatz 22 – *SQLite-Grundlagen*.

SQLite – Daten abfragen

Konzept	Beispiel
`SELECT`	`SELECT name, preis FROM produkt`
Spaltenselektion	`SELECT name` statt `SELECT *`
`WHERE`-Filter	`WHERE preis > 100`
`fetchall()`	`cursor.fetchall()`
`fetchone()`	`cursor.fetchone()[0]`

Stoff: Notebook 22 & Foliensatz 22 – *SQLite-Grundlagen*.

SQLite-Vertiefung – WHERE-Operatoren

Konzept	Beispiel
Vergleich = != < > <= >=	`WHERE anzahl >= 2`
AND	`WHERE preis > 50 AND anzahl > 1`
OR	`WHERE newsletter = 'ja' OR vip = 1`
NOT	`WHERE NOT newsletter = 'ja'`
Klammern-Gruppierung	`WHERE (a OR b) AND c`
BETWEEN	`WHERE preis BETWEEN 20 AND 50`
IN	`WHERE name IN ('Webcam', 'Funkmaus')`

Stoff: Notebook 23 & Foliensatz 23 – *SQLite-Vertiefung*.

SQLite-Vertiefung – JOINs

Konzept	Beispiel
`INNER JOIN`	`FROM bestellung INNER JOIN kunde`
`JOIN ... ON`	`ON bestellung.kunde_id = kunde.kunde_id`
Tabellen-Präfix	`kunde.name, produkt.name`
`LEFT JOIN`	`FROM kompetenz LEFT JOIN mitarbeiter`
`LEFT JOIN` + `IS NULL`	`WHERE mitarbeiter.id IS NULL` (ohne Treffer)

Stoff: Notebook 23 & Foliensatz 23 – *SQLite-Vertiefung*.

SQLite-Vertiefung – Sortieren & Aggregieren

Konzept	Beispiel
`ORDER BY ... DESC`	`ORDER BY preis DESC`
`ORDER BY ... ASC`	`ORDER BY name ASC`
`ORDER BY` mehrere Spalten	`ORDER BY stadt, name`
`COUNT()`	`SELECT COUNT(*) FROM kunde`
`SUM()`	`SELECT SUM(anzahl) FROM bestellung`
`AVG()`	`SELECT AVG(betrag) FROM gehalt`
`MIN()` / `MAX()`	`SELECT MAX(preis) FROM produkt`

Stoff: Notebook 23 & Foliensatz 23 – *SQLite-Vertiefung*.

SQLite-Vertiefung – Ändern & Löschen

Konzept	Beispiel
`UPDATE ... SET`	`UPDATE produkt SET preis = 99.0`
`UPDATE` mit `WHERE`	`UPDATE ... WHERE produkt_id = 1`
`DELETE`	`DELETE FROM bestellung`
`DELETE` mit `WHERE`	`DELETE FROM ... WHERE anzahl = 0`

Achtung: `UPDATE`/`DELETE` **immer** mit `WHERE` – sonst trifft es **alle** Zeilen!

Stoff: Notebook 23 & Foliensatz 23 – *SQLite-Vertiefung*.

Die Klausur im Überblick

Die Klausur besteht aus **drei Teilen**:

Teil	Form	Umfang
A – Multiple Choice	50 Fragen, je 1 Punkt	50 Punkte
B.1 – Programmieren	Python-Programm (Funktionen + `main()`)	30 Punkte
B.2 – Datenbank (SQL)	Python-Programm mit `sqlite3`	20 Punkte

Hilfsmittel: Erlaubt sind **ausschließlich die in Moodle hinterlegten Cheat-Sheets**. Andere Hilfsmittel sind **nicht** zugelassen.

Teil A – Multiple Choice

- **50 Fragen**, jede zählt **genau 1 Punkt** (insgesamt 50 Punkte).
- **Einzelauswahl** (genau eine richtige Antwort) **oder Mehrfachauswahl**.
- Bei **Mehrfachauswahl** gibt es **anteilige Punkte** für teilweise richtige Lösungen.
- Die Fragen decken den **gesamten In-Scope-Stoff** ab (Teil 1 & 2 dieser Folien).

So üben: Nochmal die Wiederholungsfragen aus den Moodle Tests durcharbeiten, Karteikarten bspw. Anki o.ä. - Erstellen bspw. mit KI, aber gegenchecken nicht vergessen!

Teil A – Beispielfragen (1/3)

1 · Schleifen – Wann wird eine `while`-Schleife ausgeführt?

- ✓ Solange eine Bedingung wahr ist
- ✗ Genau einmal
- ✗ Eine festgelegte Anzahl an Durchläufen
- ✗ Bis die Variable `i` den Wert 10 erreicht hat

2 · Operatoren – Der Operator and ergibt `True`, wenn …

- ✓ … beide Bedingungen `True` sind
- ✗ … mindestens eine Bedingung `True` ist
- ✗ … beide Bedingungen `False` sind

Teil A – Beispielfrage (2/3): Mehrfachauswahl

3 · Datenbanken – Welche Vorteile bieten Datenbanken gegenüber Excel? *(mehrere Antworten richtig)*

- ✓ Bessere Zugriffskontrolle und Sicherheit
- ✓ Effiziente Verknüpfung über relationale Strukturen
- ✓ Höhere Skalierbarkeit bei großen Datenmengen
- ✗ Einfachere Bedienung ohne Programmierkenntnisse
- ✗ Geringere Anschaffungskosten

Teilpunkte: Jede richtige Auswahl zählt anteilig; falsch Angekreuztes zieht ab.

Teil A – Beispielfragen (3/3)

4 · Fehlerbehandlung – Reihenfolge bei einem Fehler im `try`-Block?

- ✓ `try` (bis zum Fehler), dann `except`, dann `finally`
- ✗ `try`, dann `finally`, dann `except`
- ✗ `except`, dann `try`, dann `finally`

5 · SQL – Welche Klausel beschränkt SELECT-Ergebnisse auf Bedingungen?

- ✓ `WHERE`
- ✗ `FILTER`
- ✗ `LIMIT`
- ✗ `CONDITION`

Teil B.1 – Programmieren (Python)

Aufgabe: ein strukturiertes Python-Programm (Shop-Szenario).

Baustein	Beispiel aus der Probeklausur
Berechnende Funktion	`gesamtwert(bestaende, preise)`
Robuste Eingabe	`gueltige_zahl_einlesen()` (`while`/`try`)
Steuerung	`main()` orchestriert die Funktionen

- **Bewertung:** je Funktion eine Aufgabe + Docstring, `return` statt `print`, Arbeit über Parameter.

Umfang: ca. **doppelt** so groß wie die Probeklausur.

Teil B.2 – Datenbank (SQL)

Aufgabe: ein Python-Programm mit `sqlite3`, das eine Datenbank von Grund auf baut:

- Tabellen aus dem **ER-Modell** anlegen (**PK/FK**, 1:m-Beziehung).
- Daten **dauerhaft speichern** (`INSERT` + `commit()`).
- Fragen per SELECT beantworten, Ergebnis mit `print()` ausgeben.

Typische Frage	SQL-Mittel
Beträge über 100 €, höchste zuerst	`WHERE` + `ORDER BY ... DESC`
Kundenname je Bestellung	`INNER JOIN`

Umfang: ca. **doppelt so groß** wie die Probeklausur.

Und jetzt?

✓ Ihr kennt jetzt **genau** den Klausurstoff – Teil *Programme* und Teil *Datenbanken*.

- Geht jedes Thema im **Notebook** noch einmal aktiv durch (Beispiele selbst tippen).
- Übt das Zusammenspiel: ein **strukturiertes Programm** mit `main()` bzw. eine **komplette Datenbank** vom `CREATE TABLE` bis zur Abfrage.